Calidad y Preprocesamiento de Datos

MCIC VÍCTOR MANUEL CORZA VARGAS

MC FERNANDO AVITÚA VARELA

Unidad 5 Transformación y Preprocesamiento de Datos

Objetivo de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Identificar la necesidad de transformar los datos durante su integración
- Aplicar técnicas fundamentales como la normalización y el escalado
- Preparar los datos para análisis posteriores
- Comprender fundamentos matemáticos y casos de uso de las diferentes técnicas de preprocesamiento y transformación de datos

5.1 Transformación durante la integración de datos

La integración de datos consiste en combinar información proveniente de múltiples fuentes (bases de datos, archivos, APIs, etc.) en una vista unificada.

Estas fuentes a menudo presentan heterogeneidad: diferentes esquemas, formatos, unidades de medida, convenciones de codificación y, por supuesto, diferentes niveles de calidad.

La transformación de datos durante este proceso es fundamental para resolver estas inconsistencias y asegurar que el conjunto de datos resultante sea coherente, útil y preciso.

¿Qué es la transformación en este contexto?

Se refiere al conjunto de operaciones aplicadas a los datos extraídos de las fuentes originales antes o mientras se cargan en el destino final (por ejemplo, un Data Warehouse, un DataMart o simplemente un conjunto de datos consolidado para análisis). El objetivo es convertir los datos a un formato y estructura estandarizados y consistentes.

¿Por qué es necesaria la transformación durante la integración?

Resolver Heterogeneidad Estructural:

Diferencias de Esquema: Mapear nombres de columnas (ej. Cliente_ID vs CustomerID), ajustar tipos de datos (ej. string a integer), reordenar columnas.

Diferencias en Modelos de Datos: Convertir datos de formatos no relacionales a relacionales, o viceversa; aplanar estructuras jerárquicas.

¿Por qué es necesaria la transformación durante la integración?

Resolver Heterogeneidad Semántica:

Unidades de Medida: Convertir valores (ej. de libras a kilogramos, de Fahrenheit a Celsius).

Codificaciones y Representaciones: Estandarizar valores categóricos (ej. M/F, Hombre/Mujer, 1/0 todos a Masculino/Femenino), formatos de fecha (DD/MM/YYYY vs MM-DD-YY),etc.

Niveles de Agregación: Asegurar que los datos estén al mismo nivel de detalle (ej. ventas diarias vs ventas mensuales).

¿Por qué es necesaria la transformación durante la integración?

Limpieza y Estandarización de Valores:

Aplicar reglas de limpieza definidas previamente (ej. corregir errores tipográficos comunes).

Resolver inconsistencias que sólo son aparentes al comparar fuentes (ej. un cliente con dos direcciones diferentes en dos sistemas).

¿Por qué es necesaria la transformación durante la integración?

Adecuación para el Análisis:

Crear nuevas variables calculadas (ej. edad a partir de fecha_nacimiento).

Aplicar técnicas como la **normalización y escalado** para que los atributos numéricos sean comparables o cumplan los requisitos de ciertos algoritmos.

Retos comunes

- Identificar todas las inconsistencias necesarias a transformar.
- Definir las reglas de transformación correctas y robustas.
- Manejar grandes volúmenes de datos eficientemente durante la transformación.
- Asegurar que la transformación no introduzca nuevos errores o sesgos.

5.1.1 Normalización y escalado

Dentro de las transformaciones numéricas, la normalización y el escalado son técnicas esenciales, especialmente en el contexto del Machine Learning y el análisis estadístico.

5.1.1 Normalización y escalado

Muchos algoritmos son sensibles a la escala de las características de entrada. Por ejemplo, algoritmos basados en distancias (como **k-NN, K-Means, SVM**) o aquellos que utilizan descenso de gradiente (como **regresión lineal, redes neuronales**) pueden dar un peso desproporcionado a las características con rangos de valores más grandes, o pueden tener problemas de convergencia si las escalas son muy diferentes.

La normalización y el escalado buscan mitigar este problema ajustando el rango y/o la distribución de las variables numéricas.

Escalado

Escalado (Scaling): Término general que se refiere a cambiar el rango de los datos. No implica necesariamente un rango específico ni una distribución particular.

Métodos principales de escalado

Normalización (Normalization): A menudo se refiere específicamente a escalar los datos a un rango fijo, comúnmente [0, 1] o [-1, 1]. El método más común para esto es el **Escalado Min-Max**.

Estandarización (Standardization): Técnica específica de escalado que transforma los datos para que tengan una media (μ) de 0 y una desviación estándar (σ) de 1. También se conoce como **Escalado Z-score**.

Normalización Min-Max (Min-Max Scaling):

Objetivo: Escalar los datos a un rango fijo, típicamente [0, 1].

Fórmula: Para una característica X, cada valor x se transforma en x′ usando:

$$x' = \frac{x - \min(X)}{\max(X) - \min(X)}$$

donde min(X) y max(X) son los valores mínimo y máximo de la característica X en el conjunto de datos (o en el conjunto de entrenamiento).

Rango Resultante: [0, 1].

Normalización Min-Max (Min-Max Scaling):

Ventajas: Garantiza que todos los valores estén dentro de un rango acotado. Útil para algoritmos que requieren entradas en este rango (ej. algunas redes neuronales, procesamiento de imágenes donde los pixeles van de 0 a 255, que se puede escalar a 0-1).

Desventajas: Muy sensible a valores atípicos (outliers). Un solo valor extremo puede comprimir la mayoría de los datos en un rango muy pequeño dentro de [0, 1], perdiendo información relativa entre ellos.

Estandarización Z-score Scaling:

Objetivo: Transformar los datos para que tengan media 0 y desviación estándar 1.

Fórmula: Para una característica X, cada valor x se transforma en x′ usando:

$$x' = \frac{x - \mu}{\sigma}$$

donde μ es la media y σ es la desviación estándar de la característica X en el conjunto de datos (o en el conjunto de entrenamiento).

Rango Resultante: No está acotado a un rango fijo (los valores pueden ser positivos o negativos, y teóricamente ir de −∞ a +∞, aunque típicamente se concentran entre -3 y 3 si los datos originales se asemejan a una distribución normal).

Estandarización (Z-score Scaling / Standardization):

Ventajas: Menos sensible a outliers que Min-Max. Es el método preferido para muchos algoritmos de Machine Learning (PCA, regresión lineal/logística, SVM) que asumen que los datos están centrados en cero y/o tienen varianza unitaria. Ayuda a que los algoritmos de optimización (como el descenso de gradiente) converjan más rápido.

Desventajas: No confina los datos a un rango específico, lo cual podría ser un requisito para ciertos algoritmos.

¿Cuándo usar cuál?

Usa Normalización Min-Max si:

- Necesitas que los valores estén estrictamente en el rango [0, 1] (o [-1, 1]).
- Sabes que no hay (o has tratado) outliers significativos en tus datos.
- El algoritmo que vas a usar no hace suposiciones sobre la distribución de los datos (ej. k-NN, aunque la estandarización también funciona bien aquí).

¿Cuándo usar cuál?

Usa Estandarización (Z-score) si:

- El algoritmo asume que los datos están centrados en cero o tienen una distribución Gaussiana (aunque no es un requisito estricto, a menudo mejora el rendimiento).
- Hay outliers en los datos (es más robusta que Min-Max).
- Es el método "por defecto" o el más comúnmente recomendado para muchos algoritmos de ML (Regresión, SVM, PCA, Redes Neuronales).

Alternativa: **Escalado Robusto (Robust Scaling)**

Si los outliers son un problema significativo, se puede usar un escalador que utilice la mediana y el Rango Intercuartílico (IQR).

Fórmula:

$$x' = \frac{x - \text{mediana}(X)}{\text{IQR}(X)}$$

Es mucho menos sensible a valores extremos.

Ejemplo Práctico:

Supongamos que tenemos datos de edad y salario de 3 personas:

Persona 1: Edad = 25, Salario = 30000

Persona 2: Edad = 45, Salario = 70000

Persona 3: Edad = 60, Salario = 50000

Ejemplo Práctico:

Aplicando Normalización **Min-Max** (Rango [0, 1]):

Edad: Min=25, Max=60

Persona 1: $(25-25)/(60-25)=0/35=0$

Persona 2: $(45-25)/(60-25)=20/35\approx0.571$

Persona 3: $(60-25)/(60-25)=35/35=1$

Salario: Min=30000, Max=70000

Persona 1: $(30000-30000)/(70000-30000)=0/40000=0$

Persona 2: $(70000-30000)/(70000-30000)=40000/40000=1$

Persona 3: $(50000-30000)/(70000-30000)=20000/40000=0.5$

Ejemplo Práctico:

Aplicando Estandarización (Z-score):

Edad: Media(μ) = (25+45+60)/3 = 130/3 ≈ 43.33. Desv. Estándar(σ) ≈ 17.56

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$

Persona 1: (25−43.33)/17.56≈−1.044

Persona 2: (45−43.33)/17.56≈0.095

Persona 3: (60−43.33)/17.56≈0.949

Salario: Media(μ) = (30000+70000+50000)/3 = 150000/3 = 50000. Desv. Estándar(σ) ≈ 16330 (usando desviación estándar poblacional)

Persona 1: (30000−50000)/16330≈−1.225

Persona 2: (70000−50000)/16330≈1.225

Persona 3: (50000−50000)/16330=0

Consideraciones Importantes en la Práctica

Fuga de Datos (Data Leakage):

La fuga de datos se produce cuando la información del conjunto de pruebas influye involuntariamente en el proceso de entrenamiento, lo que conduce a un rendimiento excesivamente optimista del modelo. Para mitigar este riesgo en el proceso de transformación de datos es necesario tomar en cuenta las siguientes consideraciones:

Al preparar datos para Machine Learning, es crucial calcular los parámetros de escalado (min, max, media, desviación estándar) únicamente sobre el conjunto de entrenamiento (training set).

Luego, estos mismos parámetros se usan para transformar tanto el conjunto de entrenamiento como el conjunto de prueba (test set) y cualquier dato nuevo futuro.

Nunca calcules los parámetros usando el conjunto de prueba o todo el dataset completo antes de dividirlo.

Librerías

Librerías: Herramientas como scikit-learn en Python ofrecen implementaciones eficientes y fáciles de usar (MinMaxScaler, StandardScaler, RobustScaler).

5.1.2 Descomposición y combinación de datos

Más allá de simplemente ajustar formatos o escalas, la transformación durante la integración a menudo implica modificar la estructura misma de las características (atributos o variables) disponibles.

Descomposición de Datos

Dividir una característica compleja en múltiples características más simples.

Ejemplo 1: Una columna FechaHora (2025-04-20 13:30:00) puede descomponerse en Año (2025), Mes (4), Día (20), Hora (13), Minuto (30), Día_Semana (Domingo). Esto puede ser útil si una fuente tiene la fecha completa y otra solo el año o mes, permitiendo una integración más flexible o análisis basados en componentes temporales específicos.

Ejemplo 2: Una dirección completa (Calle Falsa 123, Colonia Centro, Ciudad Ejemplo, CP 06000) podría descomponerse en Calle, Colonia, Ciudad, CP

¿Por qué realizar estas operaciones?

- Mejorar el Rendimiento de Modelos: Las características combinadas o descompuestas pueden revelar patrones que los modelos no capturarían con las características originales. Las combinaciones polinomiales ayudan a modelar relaciones no lineales.

Combinación lineal

Implica operaciones aritméticas básicas (suma, resta, multiplicación por constantes, promedios ponderados).

Ejemplo 1: Si tenemos precio_unitario y cantidad_vendida, podemos crear ingreso_total = precio_unitario * cantidad_vendida

Ejemplo 2: Si integramos datos de ventas de dos tiendas (ventas_tienda_A, ventas_tienda_B), podemos crear ventas_totales = ventas_tienda_A + ventas_tienda_B

Combinación Cuadrática y Polinomial

Implica crear términos de mayor orden (ej. x^2,x^3,x^y). Esto es especialmente útil en modelado para capturar relaciones no lineales entre las variables.

Ejemplo 1: Para modelar la producción agrícola, podríamos usar no solo la cantidad de fertilizante, sino también fertilizante^2, ya que el efecto podría no ser lineal (demasiado fertilizante puede ser perjudicial).

Ejemplo 2: En un modelo de predicción de clics, podríamos incluir un término de interacción entre edad_usuario y tiempo_en_pagina (ej. edad_usuario * tiempo_en_pagina).

¿Por qué realizar estas operaciones?

- Crear Variables Más Significativas: Una variable combinada como `ingreso_total` puede ser más relevante para el análisis que `precio_unitario` y `cantidad` por separado.
- Facilitar la Integración: La descomposición puede ayudar a alinear datos de fuentes con diferentes niveles de detalle (ej. fecha vs. fecha-hora).

¿Por qué realizar estas operaciones?

- Reducir Complejidad (a veces): Descomponer una variable categórica compleja (ej. un código de producto con estructura interna) en sus partes puede simplificar su interpretación

Consideraciones

- La creación de demasiadas características (especialmente polinomiales) puede llevar al sobreajuste (overfitting) de los modelos.
- Requiere a menudo conocimiento del dominio para saber qué combinaciones o descomposiciones son lógicas y útiles.
- Puede aumentar la dimensionalidad del conjunto de datos (más columnas).

5.1.3 Agregación

La agregación es un proceso fundamental en la preparación de datos que consiste en sumarizar información. En lugar de trabajar con cada registro individual (instancia u observación), agrupamos los registros que comparten ciertos atributos y calculamos estadísticas resumen (como la suma, el promedio, el conteo, etc.) para cada grupo. El resultado es un nuevo conjunto de datos con menos filas (instancias), lo que representa una forma de reducción de dimensionalidad en términos de instancias.

Agregación

Se basa en dos componentes clave:

1. Atributos de Agrupación: Las columnas cuyos valores se utilizan para definir los grupos (similar al `GROUP BY` en SQL).
2. Funciones de Agregación: Las operaciones matemáticas o estadísticas que se aplican a otras columnas dentro de cada grupo. Funciones comunes incluyen:
   - `SUM()`: Suma de valores.
   - `COUNT()`: Número de instancias.
   - `AVG()` / `MEAN()`: Promedio.
   - `MIN()`: Valor mínimo.
   - `MAX()`: Valor máximo.
   - `MEDIAN()`: Mediana.
   - `STDDEV()`: Desviación estándar.

¿Por qué realizar agregación?

1. Reducción del Tamaño del Dataset: Un menor número de filas implica menor necesidad de almacenamiento y procesamiento más rápido.
2. Cambio de Granularidad: Permite pasar de un nivel de detalle fino a uno más grueso. Por ejemplo, de transacciones diarias a resúmenes mensuales o anuales. Esto es vital durante la integración si las fuentes tienen diferentes niveles de granularidad temporal o conceptual.
3. Análisis y Reportes: Muchas preguntas de negocio se responden mejor con datos agregados (ej. "¿Cuáles son las ventas totales por región?" en lugar de mirar cada venta individual).
4. Estabilidad Estadística: Los valores agregados (como promedios) tienden a ser menos ruidosos que los valores individuales.

Agregación

Consideraciones:

Se pierde información detallada irreversiblemente. Una vez agregados los datos, no se puede volver a las transacciones individuales.

La elección de las variables de agrupación y las funciones de agregación depende completamente del objetivo del análisis.

5.1.4 Generalización - Reducción de valores

La generalización es una técnica de transformación que busca simplificar los datos reemplazando valores específicos de un atributo con conceptos de nivel superior o más generales. Esto puede verse como una forma de reducción de dimensionalidad en términos de la cardinalidad (número de valores distintos) de un atributo, especialmente útil para variables categóricas, o para agrupar rangos de variables numéricas (discretización).

¿Cómo funciona?

Se basa en el uso de jerarquías conceptuales (Concept Hierarchies). Estas jerarquías definen relaciones "es un tipo de" o "es parte de" entre valores. La generalización implica "subir" en esta jerarquía.

1. Para Datos Categóricos: Reemplazar valores específicos por categorías más amplias.

    - *Jerarquía de Ubicación:* {'Calle Específica'} -> {'Colonia'} -> {'Alcaldía/Municipio'} -> {'Ciudad'} -> {'Estado'} -> {'País'}. Se podría reemplazar la calle específica por la ciudad.
    - *Jerarquía de Producto:* {'iPhone 15 Pro Max 256GB'} -> {'iPhone 15'} -> {'Smartphone'} -> {'Electrónico'}. Se podría reemplazar el modelo exacto por 'Smartphone'.

¿Cómo funciona?

1. Para Datos Numéricos (Discretización/Binning): Agrupar rangos de valores continuos en "bins" o intervalos discretos, que a menudo se tratan como categorías.

    - *Jerarquía de Edad:* Edades exactas (21, 23, 28) se reemplazan por rangos (20-29).
    - *Jerarquía de Ingresos:* Ingresos exactos ($35,000, $42,000) se reemplazan por rangos ($30k-$49k).

Ejemplo

Englobar las profesiones por áreas

ID Cliente	Edad	Ciudad	Profesión
1	28	Guadalajara	Ingeniero de Datos
2	45	Ciudad de México	Abogado
3	33	Monterrey	Médico
4	25	Ciudad de México	Diseñador Gráfico
5	52	Guadalajara	Abogado

¿Porqué realizar generalización?

1. Reducción de Cardinalidad: Simplifica atributos con demasiados valores únicos (ej. códigos postales, títulos de trabajo específicos), lo que puede mejorar el rendimiento y la interpretabilidad de algunos modelos (ej. árboles de decisión).
2. Anonimización y Privacidad: Reemplazar identificadores específicos (como una dirección exacta o fecha de nacimiento) con categorías más amplias ayuda a proteger la privacidad individual (K-anonimidad, l-diversidad).

¿Porqué realizar generalización?

3. Facilitar la Integración: Si diferentes fuentes usan distintos niveles de especificidad para un mismo concepto (ej. una fuente usa 'Ingeniero de Software' y otra usa 'TI'), se pueden generalizar ambos a una categoría común ('Profesional de TI').

4. Análisis a Mayor Nivel: Permite observar patrones en categorías más amplias en lugar de perderse en detalles específicos.

Consideraciones

- Implica una pérdida controlada de información. El nivel de generalización debe elegirse cuidadosamente.
- La definición de las jerarquías conceptuales requiere conocimiento del dominio o puede basarse en estándares.
- La generalización excesiva puede hacer que los datos sean demasiado vagos para ser útiles.

Referencias

Han, J., Pei, J., & Tong, H. (2022). Data mining: concepts and techniques. Morgan kaufmann.

Ponniah, P. (2004). Data warehousing fundamentals: a comprehensive guide for IT professionals. John Wiley & Sons.

Scikit-learn documentation. Preprocessing data https://scikit-learn.org/stable/modules/preprocessing.html